

Clustering

by Liav Samiya, Vered Shaulov, Elya Davar

Introduction

- **Why is clustering considered a form of unsupervised learning? What information is missing compared to supervised learning?**

Clustering as Unsupervised Learning:

Clustering is classified as a form of unsupervised learning because its primary objective is to discover inherent structures or hidden groups within a dataset without any external guidance.

Missing Information Compared to Supervised Learning:

The critical information missing in clustering is the "target labels". In a supervised learning scenario, the algorithm is provided with both the input data and the corresponding correct answers (labels) to learn from. In clustering, the algorithm only receives the raw data points. It must autonomously evaluate the features of each point to group similar items together and separate dissimilar ones, without knowing what those groups represent in advance.

- **What are the input and output of a clustering algorithm?**

-Input: The algorithm takes raw, multi-dimensional feature vectors representing the items you want to organize. It also relies on a similarity metric to evaluate the relationships between individual data points.

-Output: The result is a partitioned dataset divided into distinct subsets. Within these subsets, intra-cluster similarity is maximized (items in the same group are highly alike) and inter-cluster similarity is minimized (different groups are clearly distinct from one another).

Similarity and distance

- **Give an example of a situation in which Euclidean distance may not be the most natural notion of similarity.**

situation:

Document clustering or text mining

Why Euclidean Fails:

Euclidean distance is highly sensitive to vector magnitude (data length). If two documents share the exact same vocabulary and topic, but one is a short paragraph and the other is a long article, their Euclidean distance will be very large simply because of the difference in total word counts.

K-means Clustering

- **In the assignment step, what is fixed and what is being computed?**
 - What is fixed: The data points and the current locations of the cluster centroids.
 - What is computed: The cluster assignments. The algorithm calculates the distance between each data point and every centroid, then assigns the point to the closest one.
- **In the update step, what is fixed and what is being computed?**
 - What is fixed: The data points and their current cluster assignments (the specific group each point was assigned to in the previous step)
 - What is computed: The new positions of the cluster centroids. These are updated by calculating the mathematical mean (average) of all data points belonging to each respective cluster.
- **Why does K-means tend to produce clusters that are approximately spherical?**
 - The Role of Euclidean Distance: K-means relies on Euclidean distance to determine similarity. Because Euclidean distance treats all spatial directions with equal weight, any boundary defined by equal distance from a central point naturally forms a circle in 2D or a hypersphere in higher dimensions.
 - Centroid Optimization: The algorithm minimizes an objective function where every point contributes its squared distance to the nearest center. This mathematical constraint forces clusters to be as tight and compact as possible around their respective centroids.
 - Because K-means utilizes both of these components simultaneously, it is algorithmically locked into producing spherical shapes.
- **Give an example of a dataset where K-means is expected to perform poorly, and explain why.**
 - Example Dataset: A dataset where groups are stretched out tightly along a specific axis, forming long, parallel, elongated lines rather than circular clouds.
 - Why K-means fails: K-means relies on Euclidean distance, which treats variance in all spatial directions with equal weight. When a cluster is heavily elongated, points located at the far edges of the long cluster will be mathematically closer to the centroid of a neighboring cluster than to their own true center. Consequently, K-means will misclassify these outer points, fracturing the continuous line and creating artificial, rounded cluster partitions. Additionally, if the data contains strong outliers, the squared-error objective function will pull the centroids drastically toward those outliers, further distorting the boundaries.
- **Why is K-means not guaranteed to find the globally optimal clustering?**

K-means operates using a heuristic, alternating approach that splits the problem into two greedy steps: assigning points to the nearest center and recalculating those centers based on the mean. This step-by-step optimization does not look ahead to evaluate the global structure of the dataset. Since the algorithm cannot undo previous assignments that have led it down a poor path, a bad initial random state

will lock the boundaries into an inefficient layout. It will reach a point where no single move can further decrease the objective function, causing it to terminate prematurely at a local optimum.

Hierarchical Clustering

- **How does hierarchical clustering differ conceptually from K-means?**
 - K-means requires a predefined number of clusters (k) before running. Hierarchical clustering does not. It creates a full spectrum of partitions across all possible cluster sizes.
 - K-means is a top-down, iterative partitioning algorithm that optimizes point locations around moving centroids. Hierarchical clustering is usually bottom-up, sequentially merging the closest data points step-by-step.
 - K-means yields a single-level, flat partition of the dataset. Hierarchical clustering produces a multi-layered, nested tree structure called a dendrogram.
 - K-means is geometrically constrained to find compact, spherical clusters. Hierarchical clustering can adapt to irregular or elongated shapes depending on the linkage criteria used.
 - K-means is non-deterministic and sensitive to random initialization (can get stuck in local minima). Hierarchical clustering is completely deterministic, producing identical results every run.
- **What is the difference between single linkage and complete linkage?**

The fundamental difference lies in whether cluster proximity is defined by their closest neighboring points or by their most distant members.

 - Single Linkage Behavior: By focusing on the shortest path between clusters, it allows groups to merge based on local boundaries. This makes it highly flexible for capturing irregular geometries, though it can cause unrelated groups to bleed together via a sequence of close intermediate points.
 - Complete Linkage Behavior: By focusing on the furthest pair of points, it ensures that a merge only happens if all elements of both clusters are within a certain distance threshold. This enforces a strict geometric boundary, consistently yielding compact and well-separated groups
- **In what sense can average linkage be viewed as a compromise between single and complete linkage?**

Single linkage evaluates proximity based on the minimum distance between any two points in different clusters ($d(A,B)=\min d(x,y)$). Complete linkage uses the maximum distance ($d(A,B)=\max d(x,y)$). Average linkage mathematically balances both by calculating the mean of all pairwise distances between the clusters.
- **What information does a dendrogram provide?**

A dendrogram provides a complete, multi-layered visual summary of the data's nested relationships. It explicitly displays the exact history of cluster merges

alongside the mathematical distance at which each merge took place, represented by the height of the vertical links. By evaluating the tree's vertical gaps, it reveals the underlying structure of the dataset. A significant vertical jump indicates the combining of rather different groups. Ultimately, this allows a user to dynamically choose the optimal number of final clusters by identifying where to cleanly "cut" the diagram.

DBSCAN

- **What is the main idea behind DBSCAN?**

The fundamental philosophy behind DBSCAN is to identify clusters based on how tightly packed data points are in the feature space, rather than partitioning space around moving averages. The algorithm seeks out dense pockets of data that are separated from one another by sparser, low-density zones. Because it does not rely on a fixed number of centroids or spherical assumptions, DBSCAN can naturally adapt to arbitrary, highly complex shapes. Furthermore, it inherently manages anomalies by allowing isolated points that fall outside these dense regions to remain unassigned.

- **Give an example of a dataset where DBSCAN is expected to perform well.**

-Example Dataset: Mapping the spatial coordinates of stars, planets, and celestial bodies within a specific sector of a telescope's field of view. The data features dense, winding, non-convex structures like the spiral arms of a galaxy, mixed with randomly scattered background cosmic radiation and sensor anomalies

-Why DBSCAN excels: Because it tracks the continuous, snake-like density paths of the spiral arms, mapping their true geometric shape perfectly. Simultaneously, it automatically identifies the scattered background radiation as noise, keeping the final galaxy structures completely clean.

- **Give an example of a dataset where DBSCAN may fail, and explain why.**

-Example Dataset: Analyzing user interaction graphs on a social media platform (such as Twitter or Instagram). In this environment, mega-celebrities or viral global trends generate hubs of engagement with an extremely high density of data points. Meanwhile, smaller, niche interest groups (such as a local hobby club) form valid clusters but with a much more moderate density. A small number of highly active "bridge" users naturally interact across both boundaries.

-Why DBSCAN fails: This scenario presents a major challenge for DBSCAN because it combines highly variable densities with the bridging (or chaining) effect. If the neighborhood radius is configured large enough to detect and map the moderate-density local communities, the few "bridge" users will act as high-density conduits. This causes the algorithm to inadvertently chain and merge all the distinct sub-communities together with the celebrity hubs into a single, massive, meaningless cluster. Conversely, if ϵ is restricted to isolate the ultra-dense celebrity hubs

and prevent this merging, the normal, lower-density communities will fail to meet the density threshold and will be completely misclassified as noise.

The Curse of Dimensionality

- **Why does the curse of dimensionality affect both K-means and DBSCAN?**

In high-dimensional spaces, the volume of the space grows exponentially, causing data points to become extremely sparse. Crucially, the distances between all pairs of points converge and become nearly equal, completely erasing distance contrast.

-K-means: Because the relative difference between the nearest and furthest centroids approaches zero, assigning points to a specific cluster center becomes mathematically arbitrary. This invalidates the algorithm's variance minimization process.

-DBSCAN: Data sparsity destroys the concept of local density. A fixed neighborhood radius loses its discriminative power, it will either capture almost every point if expanded slightly, or label the entire dataset as noise, failing to isolate distinct clusters.

Mini Project: Exploring Clustering Algorithms

Hierarchical Clustering (Agglomerative)

Main Hyperparameters: `n_clusters`, `linkage`, `metric` (or `affinity`), and `distance_threshold`.

What Each Parameter Controls:

- `n_clusters`: Defines the granularity of the final partition (where the dendrogram is "cut").
- `linkage`: Determines the mathematical logic used to calculate the distance between clusters.
- `metric`: Defines how the distance between individual data points is computed (e.g., Euclidean, Manhattan).
- `distance_threshold`: An alternative to `n_clusters`, setting a raw numeric distance above which clusters will no longer be merged.

How Changing Parameters Affects Results:

- **Linkage**: Changing this drastically alters the cluster shape. **Ward** minimizes variance, forming compact, equal-sized clusters. **Complete** uses maximum distance, forming tight clusters but increasing outlier sensitivity. **Single** uses minimum distance, which frequently causes "chaining" (stretching points into long, unhelpful snake-like clusters).
- **Granularity**: Increasing `n_clusters` splits the data into smaller, more homogenous groups. If pushed too high, it fractures natural groupings. If set too low, distinct groups are improperly lumped together.

Potential Pitfalls: The algorithm scales poorly on large datasets due to $O(n^2)$ memory and $O(n^2 \log n)$ time complexities. Early merging mistakes are irreversible and propagate up the tree. Finally, the algorithm forces hard partitions and has no built-in outlier handling, which can severely distort Ward merges.

Reasonable Defaults & Tuning: Standardize features first. Default to `linkage='ward'` and `metric='euclidean'`. Plot a dendrogram on a representative sample of the dataset (or use tree truncation to handle the 5,000 rows) and visually cut at the largest vertical gap to determine `n_clusters` (starting around $K=3$). If cluster sizes end up highly skewed, switch to average linkage.

Gaussian Mixture Models (GMM / Expectation-Maximization)

Main Hyperparameters: `n_components`, `covariance_type`, `n_init`, and `init_params`.

What Each Parameter Controls:

- **`n_components (K)`**: The number of Gaussian distributions (clusters) to fit to the data, dictating model complexity.
- **`covariance_type`**: Controls the geometric flexibility and shape of the clusters.
- **`n_init`**: The number of times the EM (Expectation-Maximization) algorithm is randomly initialized.

- **init_params:** The strategy used to initialize the cluster centers (e.g., random vs. K-Means).

How Changing Parameters Affects Results:

- **Covariance:** Setting it to Full allows each cluster to form an unconstrained ellipse, mapping flexibly to correlated data. Changing to Diag restricts them to axis-aligned ellipses. Changing to Spherical forces circular shapes (like K-Means), which will fail to capture stretched data.
- **Components:** Too few components under-fit the data, while too many create degenerate components that collapse onto single points.
- **Initialization:** Increasing n_init ensures the algorithm avoids poor local optima, yielding more reproducible and stable clusters.

Potential Pitfalls: GMM inherently assumes clusters are roughly Gaussian, it fails on complex shapes like rings or crescents. It is also highly sensitive to its starting state. Furthermore, using a full covariance matrix scales quadratically with feature count, risking singularity errors if features are unscaled or outnumber data points.

Reasonable Defaults & Tuning: Standardize features. Use n_init=10, init_params='kmeans', max_iter=200, and covariance_type='full' (falling back to diag if matrix errors occur). Find the ideal n_components by testing K=2 to 10 and selecting the value that minimizes the Bayesian Information Criterion (BIC).

10.1 & 10.2: Practical Study & Application to the Dataset

Dataset Preparation: We applied these algorithms to the entire dataset of 5,000 rows from the provided data, isolating the numeric social media features: age, daily_screen_time_min, social_media_time_min, daily_screen_time_without_social_media_min, social_media_ratio, and negative_interactions_count. The features were standardized using StandardScaler and reduced to two dimensions via PCA to allow for visualization.

Parameter Manipulation & Clustering Changes:

- **Hierarchical Clustering Changes:** We swept K from 2 to 5 and tested all linkage types. Single linkage failed entirely, chaining data into a single mass and dropping the silhouette score to ~0.09. Ward linkage proved optimal. At K=3, Ward hit a peak silhouette score of 0.34, perfectly matching the largest merge gap in the dendrogram. Increasing clusters past 3 steadily dropped the score, indicating that cohesive natural groups were being artificially split.
- **GMM Changes:** We swept components from 2 to 5 across all covariance types. The Spherical covariance performed poorly because it assumed circular boundaries, ignoring the fact that variables like screen time and social media time scale together. Full covariance effectively mapped these elongated, correlated shapes. A model with covariance_type='full' and K=3 minimized the BIC and yielded the highest overall silhouette score (0.35).

Discussion & Visualizing the Resulting Clusters:

Both algorithms confidently converged on $K=3$. When projecting the clustered labels onto the 2D PCA space, three distinct user groups emerge:

- (1) younger heavy users with a high social media ratio
- (2) middle-aged moderate users
- (3) older users with minimal screen time.

The primary difference observed during visualization relates to hard vs. soft boundaries. Hierarchical clustering draws strict, deterministic lines between the moderate and heavy-user groups. Conversely, GMM provides probabilistic confidence per point. users located on the visual boundary of the scatter plot yield membership probabilities of roughly 0.5 to 0.6. This soft clustering realistically handles transitional behaviors that the hard hierarchical labels gloss over.